HORT 7.0 — Deep Learning Practical Task

Building a Banknote Detector with CNNs + Gradio (Phase 1)

Traitz Academy Hort 7.0

📋 Task Overview

You will build a **single banknote classifier** — upload an image of one CFA franc note, and the app predicts which denomination it is, along with a confidence score.

This is your first hands-on Deep Learning project. You'll go through the **full real-world pipeline**: collecting data → annotating it → training a CNN using transfer learning → deploying it as a working web app.

Submission Deadline: Thursday, 4th September

Submit:

- Your GitHub repo link (code + trained model)
- Your working Gradio app link (or a screen recording if hosting isn't possible)
- A short write-up (1 page) explaining your process and results

---

🎯 What You're Building

An app where:

1. A user uploads a photo of **one** CFA franc banknote
2. The app predicts the denomination (e.g., "1000 FCFA", "2000 FCFA", "5000 FCFA", "10000 FCFA")
3. The app displays a **confidence score** (e.g., "94.2% confident")

(Note: detecting multiple notes at once and totaling them is Phase 2 — not required for this task.)

🛠 Tools You'll Use (and Why)

Tool	Purpose	Why This Tool
Roboflow	Collect, organize, and label your images	Free, beginner-friendly, handles data augmentation and exports in formats ready for training — saves you from writing manual data-loading code
VS Code	Write and run your training + app code	Industry-standard code editor, works well with Python virtual environments and Git
MobileNetV2	Pretrained CNN for transfer learning	Small, fast, accurate enough for our needs, and light enough to eventually run on a phone/small server
Gradio	Turn your trained model into a web app	Minimal code needed to get a shareable, working demo — no frontend skills required
GitHub	Store and share your code	Standard practice for any real project — also how you'll submit this task

---

STEP 1: Collect Your Images

What to Do

Gather photos of CFA franc banknotes. You need **each denomination** you plan to classify — recommend starting with these 4:

- 1000 FCFA
- 2000 FCFA
- 5000 FCFA
- 10000 FCFA

Minimum target: 40-60 images **per denomination** (more is better).

How to Collect Them

- Take photos yourself using your phone camera
- Vary conditions for each note:

- Different lighting (bright, dim, natural light, indoor light)
- Different angles (straight-on, slightly tilted)
- Different backgrounds (table, hand, floor, cloth)
- Both front and back sides of each note

Why This Matters

A model only learns to recognize what it's *shown*. If all your training photos are taken in the same lighting on the same table, your model will fail the moment someone uploads a photo taken somewhere else. **Variety in your training images is what makes the model generalize to real-world conditions.**

Save all images in folders named by denomination:

```
raw_images/
  ├── 1000_fcfa/
  ├── 2000_fcfa/
  ├── 5000_fcfa/
  └── 10000_fcfa/
```

STEP 2: Annotate & Organize with Roboflow

What to Do

1. Create a free account at roboflow.com
2. Create a **new project** → choose **"Single-Label Classification"** (not object detection — we're classifying one note per image in Phase 1, not detecting multiple objects)
3. Upload all your images from Step 1
4. Roboflow will ask you to **tag each image** with its class (denomination) — since your folders are already organized by denomination, this step is quick
5. Split your dataset:
   - **70% Train** — what the model learns from
   - **20% Valid (Validation)** — used during training to check progress
   - **15% Test** *(Roboflow default splits may vary — adjust to roughly 70/20/10)*

Why Roboflow (and Why This Step Matters)

- **Consistent labeling** — manually organizing and labeling hundreds of images in code is tedious and error-prone; Roboflow handles this cleanly
- **Data augmentation** — Roboflow can automatically generate variations of your images (rotated, brightness-adjusted, flipped) to artificially grow your dataset. This directly addresses the "not enough data" problem we discussed with transfer learning
- **Train/Valid/Test split** — Roboflow does this for you automatically, avoiding a common beginner mistake of testing a model on data it already trained on (which gives falsely high accuracy)

Apply Augmentation (Recommended)

In Roboflow's "Generate" step, apply:

- Rotation (±15°)
- Brightness adjustment (±20%)
- Horizontal flip (only if note orientation doesn't matter for your use case — otherwise skip this)

Why: this simulates real-world variation without needing to physically take more photos — directly tackling the small-dataset problem.

Export Your Dataset

Export in the **Folder Structure** or **TensorFlow/Keras-compatible format**. Roboflow will give you a code snippet or a download link — save this dataset into your VS Code project folder.

STEP 3: Set Up Your Project in VS Code

What to Do

1. Create a new project folder:

```
money_detector/
  ├── dataset/          (from Roboflow export)
  ├── train_model.py
  ├── app.py
  ├── requirements.txt
  └── model/            (will hold your saved trained model)
```

2. Create and activate a virtual environment:

```bash
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # Mac/Linux
```

3. Install required packages:

```bash
pip install tensorflow gradio pillow numpy matplotlib scikit-learn
```

Why a Virtual Environment?

It keeps this project's packages isolated from other Python projects on your machine — avoiding version conflicts. This is standard practice in real development work, not just a school exercise.

STEP 4: Train Your Model with MobileNetV2 (Transfer Learning)

What to Do

Create train_model.py:

```python
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

--- Step 1: Load and prepare data ---

```
img_size = (224, 224)   # MobileNetV2's expected input size
batch_size = 16

train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    'dataset/train',
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)

val_generator = val_datagen.flow_from_directory(
    'dataset/valid',
    target_size=img_size,
    batch_size=batch_size,
    class_mode='categorical'
)

num_classes = train_generator.num_classes
print("Classes found:", train_generator.class_indices)
```

 # --- Step 2: Load MobileNetV2 (pretrained, without its original top layer) ---

```
base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,       # exclude the original 1000-class ImageNet output layer
    weights='imagenet'       # use pretrained weights
)

# Freeze the base model - we don't want to retrain these layers yet
base_model.trainable = False
```

 # --- Step 3: Add our own classification head ---

```
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation='relu')(x)
predictions = Dense(num_classes, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=predictions)
```

 # --- Step 4: Compile the model ---

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',   # same log-loss concept from Logistic Regressio
    metrics=['accuracy']
)
```

 # --- Step 5 & 6: Train and Export Model ---

```
# --- Step 5: Train ---
history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=15
)

# --- Step 6: Save the trained model ---
model.save('model/banknote_classifier.h5')
print("Model saved successfully!")
```

Why Each Part Matters

- **include_top=False** — we strip off MobileNetV2's original output layer (which was built for 1000 ImageNet categories) since we only need 4 categories (our denominations)
- **base_model.trainable = False** — this **freezes** the pretrained layers, so we only train our new final layers. This is the essence of transfer learning: reuse what the model already knows about general image features (edges, shapes, textures), and only teach it what's specific to our task

- **GlobalAveragePooling2D** — condenses the feature maps into a manageable size before the final classification layers
- **categorical_crossentropy** — this is the multi-class version of the log-loss cost function you learned in Logistic Regression
- **epochs=15** — start here; if accuracy is still improving by epoch 15, you can increase this

After Training — Check Your Results

Plot training vs validation accuracy/loss to check for **overfitting or underfitting** (a concept from our last lesson):

```python
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training vs Validation Accuracy')
plt.savefig('training_plot.png')
plt.show()
```

What to look for:

- If training accuracy is high but validation accuracy is much lower → **overfitting** (model memorized training images)
- If both are low → **underfitting** (model hasn't learned enough — try more epochs or unfreezing some base layers)

STEP 5: Build the Gradio App

What to Do

Create app.py:

Install the Dependencies you will use E.g gradio

```python
import gradio as gr
import tensorflow as tf
import numpy as np
from PIL import Image

# --- Load the trained model ---
model = tf.keras.models.load_model('model/banknote_classifier.h5')

# --- Class labels (must match the order from training) ---
class_names = ['1000_fcfa', '2000_fcfa', '5000_fcfa', '10000_fcfa']  # update to match

def predict(image):
    # Preprocess the uploaded image to match training format
    image = image.resize((224, 224))
    img_array = np.array(image) / 255.0
    img_array = np.expand_dims(img_array, axis=0)  # add batch dimension

    # Run prediction
    predictions = model.predict(img_array)[0]
    predicted_class = class_names[np.argmax(predictions)]
    confidence = float(np.max(predictions)) * 100

    return f"{predicted_class} — {confidence:.2f}% confidence"

# --- Build the Gradio interface ---
demo = gr.Interface(
    fn=predict,
    inputs=gr.Image(type="pil"),
    outputs="text",
    title="CFA Franc Banknote Detector",
    description="Upload a photo of a single CFA franc banknote to identify its denomir
)

demo.launch()
```

Why Each Part Matters

- **Resizing to 224×224** — MobileNetV2 was trained expecting this exact input size; feeding a different size will cause an error
- **Normalizing (/255.0)** — matches the preprocessing we did during training (rescale=1./255) — the model expects the same scale of input it was trained on
- **np.expand_dims** — the model expects a *batch* of images, even if you're only predicting one; this adds that extra dimension
- **np.argmax** — picks the class with the highest probability (this is your prediction)
- **Gradio's gr.Interface** — handles building the entire web UI (upload button, output display) with almost no code

Run It Locally First

```bash
python app.py
```

This will give you a local URL **(e.g., http://127.0.0.1:7860)** — open it in your browser and test uploading a few banknote photos.

STEP 6: Push to GitHub

What to Do

1. Create a .gitignore file to avoid uploading unnecessary large files:
2. Copy the shot below into the .gitignore file

```
venv/
__pycache__/
dataset/
```

(Your dataset can be large — keep it out of GitHub, or use Git LFS if you want to include it)

3. Initialize and push your repo:

```bash
git init
git add .
git commit -m "Phase 1: Banknote classifier with MobileNetV2 and Gradio"
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Why GitHub

Every real ML/software project needs version control and a shareable code location — this is non-negotiable practice, not optional polish. It also lets us (and future you) review your work.

✅ Submission Checklist

- [ ] Collected and organized images for at least 4 banknote denominations (40-60 images each minimum)
- [ ] Annotated and split the dataset using Roboflow
- [ ] Applied at least one form of data augmentation in Roboflow
- [ ] Trained a MobileNetV2-based model using transfer learning
- [ ] Plotted training vs validation accuracy and briefly commented on overfitting/underfitting in your write-up

- [ ] Built a working Gradio app that takes an image and returns prediction + confidence
- [ ] Pushed complete code to GitHub (model file, training script, app script)
- [ ] Submitted your GitHub link + Gradio app link (or demo recording) + 1-page write-up

---

💡 **Notes Before You Start**

- **Don't skip Step 1 (data collection).** The single biggest reason ML projects fail is bad or insufficient data — not a bad model. Spend real time here.
- If your validation accuracy is poor, first check your data (enough images? good variety?) before assuming the model architecture is the problem.
- If you can't get Gradio hosted publicly (e.g., via Hugging Face Spaces), a **screen recording of the app running locally** is an acceptable submission — just say so in your write-up.
- Keep your write-up short and honest — mention what worked, what didn't, and what you'd improve with more time. We care more about your understanding of the process than a perfect result.

Good luck — this is your first full Deep Learning deployment. Take it step by step, and don't hesitate to bring back your training_plot.png results if you want help interpreting whether you're overfitting or underfitting before submission.